人工智能原理与应用

- **人工智能原理与应用**
  - 人工智能简介
  - 数学基础
- **知识系统**
  - 确定性知识系统
    - 表示方法
      - 1. 谓词逻辑表示法
      - 2. 产生式表示法
      - 3. 语义网络表示法
      - 4. 框架表示法
    - 推理方法
  - 不确定性知识系统
    - 可信度推理模型 CF，$Certainty\ Factor$
    - 主观Bayes推理
    - 证据理论 $Dempster-Shafer$
    - 模糊推理
      - 分类
      - 合成 $R_1 \circ R_2$
      - 模糊修饰语四种主要运算
      - 语义距离
      - 贴近度
      - 模糊关系
    - 概率推理
- **智能搜索技术**
  - 状态空间的启发式搜索
    - 概念
    - A 算法
    - A* 算法
  - 与/或树的启发式搜索
    - 概念
    - 解树的代价
  - 博弈树的启发式搜索
    - 极大极小过程
    - α-β剪枝
  - 进化搜索
    - 蚁群算法
    - 遗传算法 Genetic Algorithm
    - 粒子群优化算法 particle swarm optimization, PSO
      - 粒子速度和位置的更新
      - 交换子和交换序
      - 解决TSP问题的速度算式
      - 算法流程
      - 与遗传算法比较
        - 共性
        - 差异
- **连接学习**
  - 人工神经网络的分类
  - 深度卷积神经网络
    - 卷积操作 (Convolution)

- 稀疏连接 (Sparse Connectivity)
- 权值共享 (Weight Sharing)
- 池化操作 (Pooling Operation)
- 连接学习
  - Hebb学习规则
  - 纠错学习规则
  - *竞争学习规则*
  - *随机学习规则*
- 单层感知器学习算法

卷积神经网络 RNN
- 结构
  - 结构特性
- 损失函数
  - **平均绝对误差损失函数**（Mean Absolute Error）
  - **均方差损失函数**（Mean Squared Error）
- 结构
- 应用

循环神经网络 RNN
- 结构图
- 结构
- 原理
- 计算方法
- 模式
  - 序列到类别的模式
  - 同步的序列到序列模式
  - 异步的序列到序列模式
- 双向循环神经网络
- 长短时记忆网络模型 Long Short-term Memory Network
  - LSTM 门控制机制
    - Forget Gate 遗忘门
    - Input Gate 输入门
    - Memory Cell 记忆元
    - Output Gate 输出门

人工智能简介

ChatGPT : Chat Generative Pre-trained Transformer

人工智能 :

- 用人工的方法在机器上实现的智能(机器智能)
- 模拟、延伸、扩展人类的智能

三大学派 :

- · 符号主义学派 :
  - 逻辑主义、心理学派
  - 以知识为基础、功能模拟
  - e.g. 专家系统
- · 连接主义学派 :
  - 仿生/生理学派
  - 以神经元为基础、结构模拟
  - 计算智能 - 神经网络

e.g. AlphaGo - 深度强化学习 + 蒙特卡洛搜索

· 行为主义学派 :

进化主义、控制论

以感知和行动为基础、行为模拟

[[人工智能原理与应用/机器学习|机器学习]]

符号主义机器学习

连接主义机器学习

数学基础

向量的范数

性质

知识系统

知识系统 : 基于**知识表示**和**知识推理**所形成的智能系统

确定性知识系统

演绎推理不能增殖新知识, **归纳推理**可以增殖新知识

表示方法

1. 谓词逻辑表示法

- 谓词
  用来表示谓词逻辑中命题，形如 $P(\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_n)$。其中P是谓词名，表示**个体的性质、状态或个体之间的关系**；$\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_n$ 是个体，即命题的主语，表示独立存在的事物或概念。
  1. 谓词是D 到 $\{T, F\}$ 的映射，函数是D到D的映射（D是个体域）
  2. 谓词的真值是T和F，函数的值 (无真值) 是D中的元素
  3. 谓词可独立存在，函数只能作为谓词的个体
- 连词
  1. ∨：“**析取**”，表示所连结的两个命题之间具有“或”的关系
  2. ∧：“**合取**”， 表示所连结的两个命题之间具有“与”的关系
  3. →：“**条件**”或“**蕴舍**”，表示“若...则...”，读作“如果P，则Q”。其中，P称为条件的前件，O称为条件的后件。
  4. ↔：“**双条件**”，表示“当且仅当”，$\boldsymbol{P} \leftrightarrow \boldsymbol{Q}$读作“P当且仅当Q”
- 量词
  全称量词
  存在量词
- 辖域: 位于量词后面的单个谓词或者用括弧括起来的合式公式
- 约束变元: 辖域内与量词中同名的变元
- 自由变元: 不受约束的变元
- 表示步骤
  1. 先根据要表示的知识定义谓词

2. 再用连词、量词把这些谓词连接起来

- 机器人移盒子
  问题的初始状态
  `AT(robot,c)；EMPTY(robot)；ON(box,a)；TABLE(a)；TABLE(b)`
  - 问题的目标状态
    `AT(robot,c)；EMPTY(robot)；**ON(box,b)**；TABLE(a)；TABLE(b)`
  - 各操作的条件和动作
    `Goto(x, y)`
    - 条件：`AT(robot, x)`
    - 动作：删除表：`AT(robot, x)`；添加表：`AT(robot, y)`
      `Pickup(x)`
    - 条件：`ON(box, x)，TABLE(x)，At(robot, x)，EMPTY(robot)`
    - 动作：删除表：`EMPTY(robot)，ON( box, x)`；添加表：`HOLDS(robot, box)`
      `Setdown(x)`
    - 条件：`AT(robot, x)，TABLE(x)，HOLDS(robot, box)`
    - 动作：删除表：`HOLDS(robot, box)`；添加表：`EMPTY(robot)，ON(box, x)`
  - 求解过程
    - 开始：`Goto(c, a)；Pickup(a)`
    - 状态1(初始状态)：`AT(robot, c)；EMPTY(robot)；ON(box, a)；TABLE(a)；TABLE(b)`
    - 状态2: `AT(robot, a)；EMPTY(robot)；ON(box, a)；TABLE(a)；TABLE(b)`
    - 状态3: `AT(robot, a)；HOLDS(robot, box)；TABLE(a)；TABLE(b)；Goto(a, b)；Setdown(b)；Goto(b, c)`
    - 状态4: `AT(robot, b)；HOLDS(robot, box)；TABLE(a)；TABLE(b)`
    - 状态5: `AT(robot, b)；EMPTY(robot)；ON(box, b)；TABLE(a)；TABLE(b)`
    - 状态6(目标状态)：`AT(robot, c)；EMPTY(robot)；ON(box, b)；TABLE(a)；TABLE(b)`

2. 产生式表示法

事实：三元组
规则：前提条件 + 结论 `IF……AND……THEN……`

3. 语义网络表示法

- 弧：表示所连两个实体之间的语义联系，必须带有标识
- 语义基元：三元组表示为 (节点1，弧，节点2)
- 基本网元：一个语义基元对应的有向图，语义网络中最基本的结构单元
- 推理过程：继承、匹配
- 基本语义关系
  - 类属关系
    属性**有继承性**，即处在具体层的节点可以继承抽象层节点的所有属性
    - 实例关系：ISA

体现“具体与抽象”的概念，含义为“是一个”，表示一件事物是另一件事物的一个实例
    - 分类关系：AKO
      泛化关系，体现“子类与超类”的，含义为“是一种”，表示一个事物是另一个事物的一种类型
    - 成员关系：$A-Member-of$
      体现“个体与集体”的关系，含义为“是一员”，表示一个事物是另一个事物的一个成员
  - 聚类关系
    - 属性关系：$Have, Can$
    - 包含关系（聚类关系）
      指具有组织或结构特征的“部分与整体”之间的关系，聚类关系一般不具备属性的继承性。如$Part-of$。
  - 时间关系：$Before$，$After$
  - 位置关系：$Located-on/under/at$
  - 相近关系：$Similar-to$，$Near-to$

4. 框架表示法

推理方法

1. **自然演绎推理**：P规则（前提引入）、T规则（结论引用）、假言推理、取拒式推理、全称固化
2. **鲁滨逊归结原理/消解原理**：去掉子句中互补的部分后析出，求归结式，若空为NIL，得出原几个子句无法同时成立

不确定性知识系统

表示：概率、可信度（大于0接近真，小于0接近假）、隶属度
原因
所需知识不完备、不精确，所需知识描述模糊
多种原因导致同一结论、解题方案不唯一

可信度推理模型 CF，$Certainty\ Factor$

模型：`IF E THEN H (CF(H,E))` $CF(H,E)=\begin{cases}\frac{P(H|E)-P(H)}{1-P(H)}, & 若\ P(H\mid E)>P(H),\\ 0, & 若\ P(H\mid E)=P(H),\\ -\frac{P(H)-P(H|E)}{P(H)}=\frac{P(H|E)-P(H)}{P(H)}, & 若\ P(H\mid E)<P(H).\end{cases}$

$CF(H,E)=MB(H,E)-MD(H,E)$

信任增长度 MB：$MB(H,E)=\begin{cases}1, & \text{if } P(H)=1,\\ \dfrac{\max\{P(H\mid E),\,P(H)\}-P(H)}{1-P(H)}, & \text{otherwise.}\end{cases}$ 不信任增长度

MD：$MD(H,E)=\begin{cases}1, & \text{if } P(H)=0,\\ \dfrac{\min\{P(H\mid E),\,P(H)\}-P(H)}{-P(H)}, & \text{otherwise.}\end{cases}$

对H的信任增长度等于对非H的不信任增长度，对H的可信度与非H的可信度之和等于0
不确定性的更新公式 $CF(H)=CF(H,E)\times max\{0,CF(E)\}$

主观Bayes推理

模型：`IF E THEN (LS,LN)H`
概念
知识强度 (LS, LN)
充分性度量 $LS = \frac{P(E|H)}{P(E|\neg H)}$
必要性度量 $LN = \frac{P(\neg E|H)}{P(\neg E|\neg H)} = \frac{1-P(E|H)}{1-P(E|\neg H)}$
几率函数
$O(X) = \frac{P(X)}{1-P(X)} = \frac{P(X)}{P(\neg X)}$
$O(H \mid E) = LS \cdot O(H)$
$O(H \mid \neg E) = LN \cdot O(H)$

证据理论 $Dempster - Shafer$

幂集：设$\mathbf{\Omega}$为样本空间，且$\mathbf{\Omega}$中的每个元素都相互独立，则由$\mathbf{\Omega}$的所有子集构成的幂集记为$\mathbf{2^{\Omega}}$，元素个数为$\mathbf{2}^N$
概率分配函数：$m: 2^{\Omega} \to [0,1]$，$m(\varnothing) = 0$，$\sum_{A \subseteq \Omega} m(A) = 1$
$m: 2^{\Omega}$上的概率分配函数
$m(A): A$的基本概率数

模糊推理

分类

1. **模糊拒取式推理**：`IF x is F THEN y is G` 若有 V 上的一个模糊集 G'，且 G 可以和 G' 匹配，则可以推出 $x\ is\ F'$，且 F' 是 上的一个模糊集，$F' = R'\ o\ G$
2. **模糊假言三段论推理**

合成 $R_1 \circ R_2$

最小-最大"合成： 设 R_1 与 R_2 分别是 $U \times V$ 与 $U \times W$ 上的两个模糊关系，则 R_1 与 R_2 的合成是从 U 到 W 的一个模糊关系，其隶属函数为$\mu_{R_1 \circ R_2}(u,w) = \bigvee_{v \in V}\{\mu_{R_1}(u,v) \wedge \mu_{R_2}(v,w)\}$"
计算方法： 把 R_1 的第 i 行元素分别与 R_2 的第 j 列的对应元素相比较，两数中取最小者，然后在这一组最小数中再取最大者，作为 $R(i,j)$

模糊修饰语四种主要运算

1. 求补：表示否定，如"不"、"非"等，$\mu_{非F}（u) = 1 - \mu_F（u）u\epsilon[0,1]$
2. **集中**：表示"很"、"非常"等，减少隶属函数的值：$\mu_{非常F}（u) = \mu_{F^2}（u）u\epsilon[0,1]$
3. **扩张**：表示"有些"、"稍微"等，增加隶属函数的值：$\mu_{有些F}（u) = \mu_F 1/2（u）u\epsilon[0,1]$
4. 加强对比：表示 "明确"、"确定"等，增加0.5以上隶属函数的值，减少 0.5以下录属函数的值：
$$\mu_{确实F}(u) = \begin{cases} 2\,\mu_F^2(u), & 0 \le \mu_F(u) \le 0.5, \\ 1 - 2\,(1-\mu_F(u))^2, & 0.5 \le \mu_F(u) \le 1. \end{cases}$$

语义距离

用于刻画两个模糊概念之间的差异，主要讨论汉明距离$Hamming\ distance$

离散论域：$d(F,G)=\frac{1}{n}\sum_{i=1}^{n}|\mu_F(u_i)-\mu_G(u_i)|.$

连续论域：$d(F,G)=\frac{1}{b-a}\int_a^b|\mu_F(u)-\mu_G(u)|\,du.$

贴近度

$(F,G)=\frac{1}{2}(F\cdot G)+\big(1-F\circ G\big)$

内积：$F\cdot G=\bigvee_{u_i\in U}(\mu_F(u_i)\wedge\mu_G(u_i))$

外积：$F\circ G=\bigwedge_{u_i\in U}(\mu_F(u_i)\vee\mu_G(u_i))$

模糊关系

$$R_m=\int_{U\times V}\Big((\mu_F(u)\wedge\mu_G(v))\vee\big(1-\mu_F(u)\big)\Big)\,\mathrm{d}(u,v)$$
$$R_c=\int_{U\times V}\big(\mu_F(u)\wedge\mu_G(v)\big)\,/(u,v)$$
$$R_g=\int_{U\times V}\big(\mu_F(u)\to\mu_G(v)\big)\,/(u,v)$$
$$\mu_F(u)\to\mu_G(v)=\begin{cases}1, & \mu_F(u)\le\mu_G(v),\\ \mu_G(v), & \mu_F(u)>\mu_G(v).\end{cases}$$

概率推理

全联合概率分布：$P(x_1,\ldots,x_n)=P(x_1)\prod_{i=2}^{n}P(x_i\mid x_1,\ldots,x_{i-1})$

联合概率分布：$P(\mathbf{x})=\prod_{k=1}^{K}P(x_k\mid x_{\pi_k})=\prod_{i=1}^{n}P\big(x_i\mid\mathrm{par}(X_i)\big)$

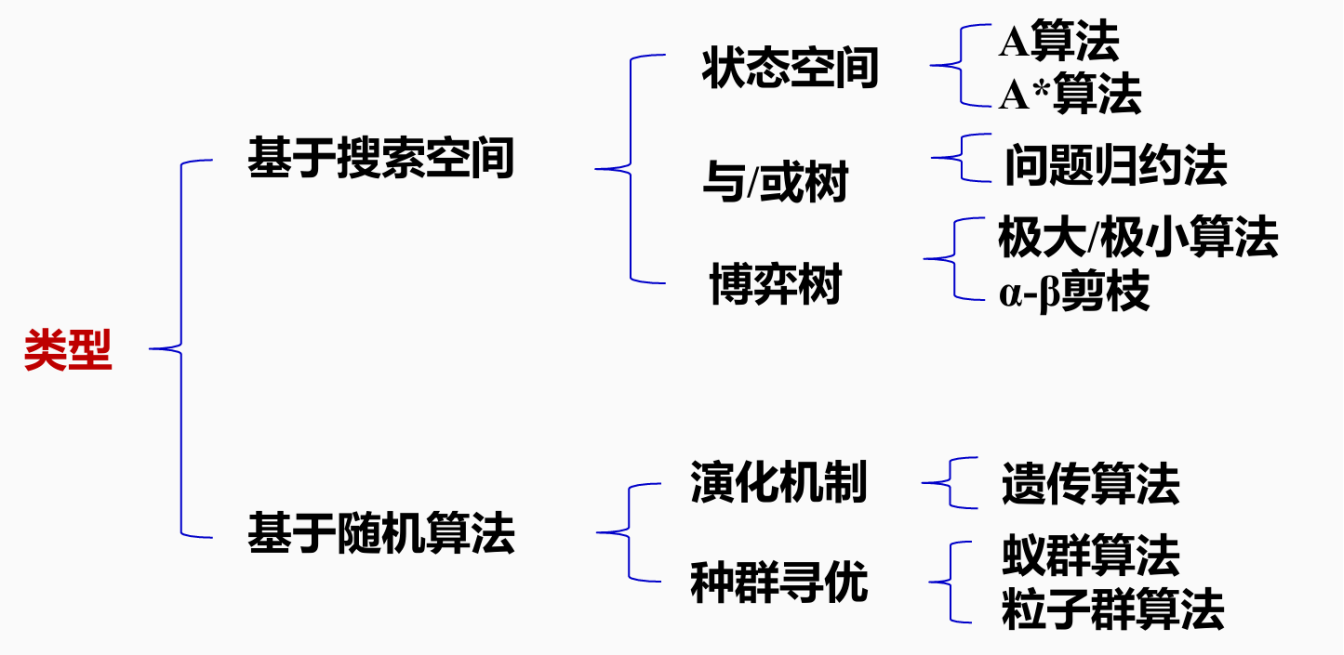
精确推理：基于枚举、变量消元、基于团树传播的算法

近似推理：$Markov\ Chain\ Monte\ Carlo, MCMC$ 算法

智能搜索技术

搜索：依靠经验，利用已有知识，根据问题的实际情况，不断寻找可利用知识构造一条**代价最小**的**推理路线**，使问题得以解决。

智能搜索：利用搜索过程得到的**中间信息**来引导搜索项往最优方向发展。



状态空间的启发式搜索

概念

- **状态**($State$)：问题求解过程中每一步问题状况的数据结构$\mathbf{S_k} = \{\mathbf{S_{k0}}, \mathbf{S_{k1}}, \ldots\}$
- **操作**($Operator$)：算符，把问题从一种状态变换为另一种状态。可是一个机械步骤，一个运算，一条规则或一个过程。可理解为状态集合上的一个函数，它描述了状态之间的关系。
- **状态空间**($Statespace$)：一个问题的全部状态以及这些状态之间的相互关系。常用一个三元组表示为：$(\mathbf{S}, \mathbf{F}, \mathbf{G})$，S为问题的所有初始状态集合；F为操作的集合；G为目标状态的集合。
- **状态空间图**：赋值的有向图，节点表示问题的状态，有向边表示操作。
- **启发性信息**：与具体问题求解过程有关，并可指导搜索过程朝着最有希望方向前进的控制信息。启发能力越强，扩展的无用结点越少。
- **估价函数**：估计节点重要性，从初始节点 S_0 出发，约束经过节点 n 到达目标节点 S_g 的所有路径中最小路径代价的估计值 $\mathbf{f(n) = g(n) + h(n)}$
  - $\mathrm{g(n)}$ 是从初始节点 S_0 到节点 n 的实际代价
  - $\mathrm{h(n)}$ 是从节点 n 到目标节点 S_g 的最优路径的估计代价

A 算法

每一步都利用估价函数 $f(n) = g(n) + h(n)$ 对 $Open$ 表中的节点进行排序

- 全局择优：从 $Open$ 表的**所有节点**中选择一个估价函数值最小的进行扩展
- 局部择优：仅从**刚生成的子节点**中选择一个估价函数值最小的进行扩展
- 八数码难题：$\mathbf{f(n) = d(n) + W(n)}$
  - $d(n)$ 表示节点n在搜索树中的深度
  - $W(n)$ 表示节点n中“不在位”的数码个数

A* 算法

对A算法（全局择优）中的 $g(n)$ 和 $h(n)$ 分别提出如下限制：

1. $\mathrm{g(n)}$ 是对最小代价 $\mathrm{g^*(n)}$ 的估计, 且 $\mathrm{g(n)} > 0$
2. $h(n)$ 是最小代价 $h*(n)$ 的下界，即对任意节点 n 均有 $h(n) \leq h*(n)$

- **可纳性**：对任一状态空间图，当从初始节点到目标节点有路径存在时，如果搜索算法总能在有限步骤内找到一条从初始节点到目标节点的最佳路径，并在此路径上结束，则称该搜索算法是可采纳的
- **最优性**：$A*$ 算法的搜索效率很大程度上取决于启发函数 $h(n)$。一般来说，在满足 $h(n) \leq h*(n)$ 的前提下，$h(n)$ 的值越大越好，携带的启发性信息越多，$A*$ 算法搜索时扩展的节点就越少，搜索效率就越高。

与/或树的启发式搜索

概念

- **本原问题**：那种不能（或不需要）再进行分解或变换且可直接解答的问题。本原问题所对应的节点称为**终止节点**。
- **端节点**：在与/或树中，**没有子节点**的节点。终止节点一定是端节点，但端节点不一定是终止节点。
- 可解节点与不可解节点
- **解树**：问题归约求解过程就实际上就是生成解树，即证明原始节点是可解节点的过程。
- **希望树**：搜索过程中最有可能成为最优解树的那棵树，与/或树的启发式搜索过程就是不断地选择、修正希望树的过程，在过程中，希望树不断变化。

解树的代价

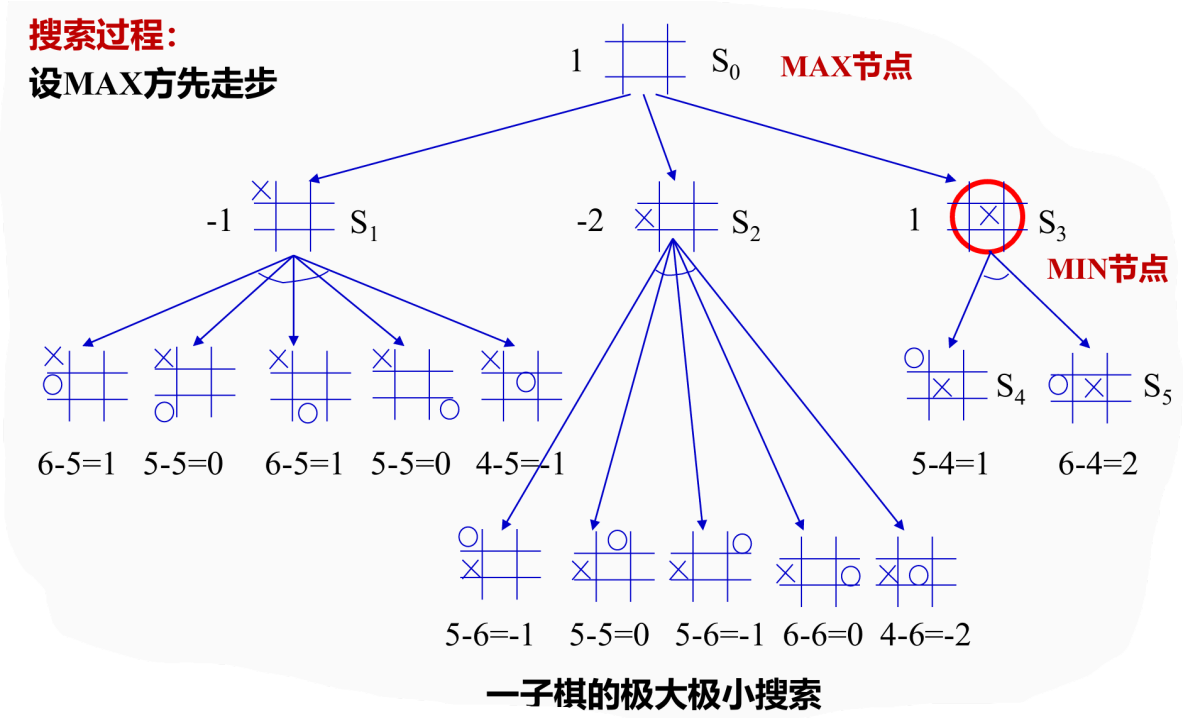
1. 若n为**终止节点**，则其代价$h(n)=0$
2. 若n为**或节点**, 且子节点为 $n_1, n_2, \ldots, n_k$, 则n的代价为：$h(n)=\min_{1 \leq i \leq k}\left\{c\left(n, n_i\right)+h\left(n_i\right)\right\}$ 其中，$c(n, n_i)$ 是节点 n 到其子节点 n_i 的边代价
3. 若n为**与节点**, 且子节点为 $\mathrm{n}_1, \mathrm{n}_2, \ldots, \mathrm{n}_{\mathrm{k}}$, 则n的代价可用和代价法或最大代价法
   - **和代价法** $h(n)=\sum_{i=1}^k\{c(n, n_i)+h(n_i)\}$
   - **最大代价法** $h(n)=\max_{1 \leq i \leq k}\left\{c\left(n, n_i\right)+h\left(n_i\right)\right\}$
4. 若n是**端节点**, 但又不是终止节点, 则n不可扩展, 其代价定义为 $\mathrm{h}(\mathbf{n})=\infty$
5. 根节点的代价即为解树的代价

博弈树的启发式搜索

- 博弈树：表示双人完备信息博弈过程的与/或树，下一步该MAX走步的节点称为MAX节点，下一步该MIN走步的节点称为MIN节点。
  1. 博弈的初始状态是初始节点
  2. 博弈树中的“或”节点和“与”节点是逐层**交替出现**
  3. 整个博弈过程始终站在某一方的立场上，例如MAX方。所有能使自己一方获胜的终局都是本原问题，相应的节点是可解节点；所有使对方获胜的终局都是不可解节点。

极大极小过程

- 对**叶节点**的估值方法是：那些对MAX有利的节点，其估价函数取正值；那些对MIN有利的节点，其估价函数取负值；那些使双方均等的节点，其估价函数取接近于0的值。
- 为**非叶节点**的值，必须从叶节点开始向上倒推。
  1. 对于MAX节点，由于MAX方总是选择估值最大的走步，因此，MAX节点的倒推值应该取其后继节点估值的最大值。
  2. 对于MIN节点，由于MIN方总是选择使估值最小的走步，因此MIN节点的倒推值应取其后继节点估值的最小值。
  3. 这样一步一步的计算倒推值，直至求出初始节点的倒推值为止。



一子棋的极大极小搜索

α-β剪枝

剪枝方法

MAX节点（或节点）的α值为当前子节点的最大倒推值

MIN节点（与节点）的β值为当前子节点的最小倒推值

β剪枝：任何MAX节点n的α值大于或等于它先辈节点的β值，则n 以下的分枝可停止搜索，并令节点n 的倒推值为α。

α剪枝：任何MIN节点n的β值小于或等于它先辈节点的α值，则n 以下的分枝可停止搜索，并令节点n的倒推值为β。

进化搜索

引入生物进化过程中的 繁殖（$Reproduction$） 变异（$Mutation$） 竞争（$Competition$） 选择（$Selection$）在达尔文进化论和孟德尔遗传变异理论的基础上产生的一种在**基因**和**种群层次**上模拟自然界生物进化过程与机制进行问题求解的**自组织、自适应**的**随机搜索**技术。

- **遗传算法**（Genetic Algorithm，GA）
- **进化策略**（Evoluenettionary Strategy, ES）
- **进化规划**（Evolutionary Programming, EP）
- **遗传规划**（Genetic Programming, GP）

蚁群算法

蚁群优化（ant colony optimization, ACO）

α、β分别表示信息素和启发式因子的相对重要程度, m为蚂蚁数

ant-cycle（蚁周）ant-quantity（蚁量）ant-density（蚁密）、

- α = 0，蚂蚁之间无协同作用；α = 1，有协同作用
- n=1m—1.5m 时，ant-cycle可以在有较好的**全局收敛性**和**收敛速度**
  初始化$\tau = \tau_0 \rightarrow$ *多蚂蚁按*$P(i \rightarrow j)$*构造解* $\rightarrow$ *蒸发与增益* $\rightarrow$ *记录最优*
  目标（TSP示例）：$L = \sum_i d_{\pi_i,\pi_{i+1}}$，$\eta_{ij} = \frac{1}{d_{ij}}$
  关键量：信息素 τ_{ij}，启发式 η_{ij}，权重 α, β，蒸发率 ρ，增量尺度 Q

转移概率（从节点i选下一节点j，候选集合J_i）：$P(i \to j) = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{k \in J_i} [\tau_{ik}]^\alpha [\eta_{ik}]^\beta}$

全局蒸发：$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij}$

信息素增益（AS/通用）：$\tau_{ij} \leftarrow \tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{(k)}$

个体贡献：$\Delta\tau_{ij}^{(k)} = \begin{cases} \frac{Q}{L_k}, & \text{若边 } (i,j) \text{ 属于蚂蚁 } k \text{ 的解} \\ 0, & \text{其它} \end{cases}$

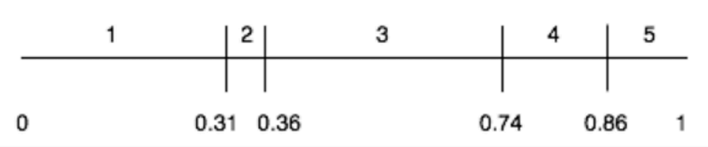
遗传算法 Genetic Algorithm

一般步骤：

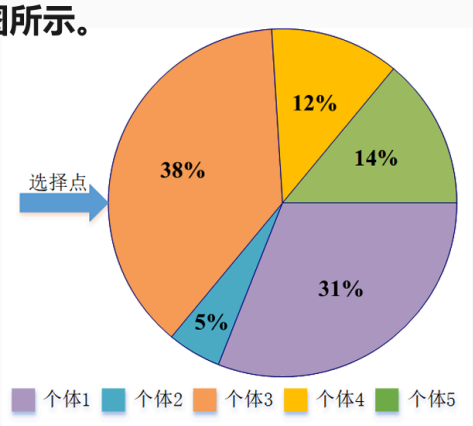
1. 随机产生种群
2. 根据策略判断个体的**适应度**，是否符合优化准则，若符合，输出最佳个体及其最优解，结束。否则，进行下一步。一般是**目标函数**经过**尺度变换**得到。
3. 依据适应度**选择**父母，适应度高的个体被选中的概率高，适应度低的个体被淘汰。
   轮盘赌选择法 (rolette wheel selection)

   首先将个体适应度值映射到轮盘中，个体的适应度值越大，其在轮盘中分配到的角度就越大，因此被选中的概率就越大。

   假设有5个个体，各适应度值分别为[6.82,1.11,8.48,2.57,3.08]，总适应度值为22.06。分别计算各个适应度值占总适应度值之比，即为[0.31,0.05,0.38,0.12,0.14]。适应度值占比示意图和轮盘赌选择示意图分别如下图所示。

   适应度值占比示意图

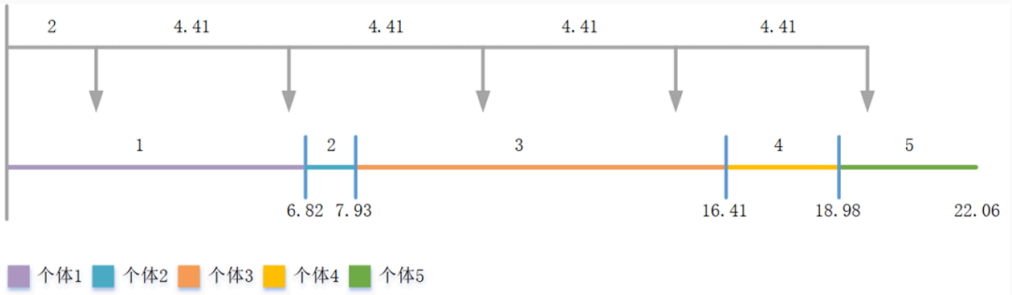
   

   轮盘赌选择示意图

   随机遍历抽样法 (stochastic universal sampling)

如果需要选择N个个体，则只需一次生成N个等间距的标记指针位置，即可选择出N个个体。假设总适应度值为F，选择个体数目为N，则SUS具体步骤如下：

- **STEP1：计算指针的间距P=F/N；**
- **STEP2：随机生成起点指针位置Start=[0~P之间的随机数]；**
- **STEP3：计算各指针的位置Pointers=[Start+i*P(其中i=[0,1,...N-1])]；**
- **STEP4：根据各指针位置，选择出N个个体。**



截断选择法（truncation selection）

个体按适应度排列，只有优秀个体能够成为父个体，参数为截断阈值（选作父体的比）。

锦标赛选择法 (tournament selection)

随机从种群中挑选一定数目个体（竞赛规模），其中最好的个体作为父个体，此过程重复进行完成个体的选择。

4. 用父母的染色体按照一定的方法进行**交叉**，生成子代

实值重组

离散重组：子个体的每个变量可以按等概率随机地挑选父个体。

中间重组：子个体 = 父个体 $1+\alpha\times$ (父个体 2 - 父个体 1)

α 是比例因子, 由 $[-d, 1+d]$ 上均匀分布地随机数产生

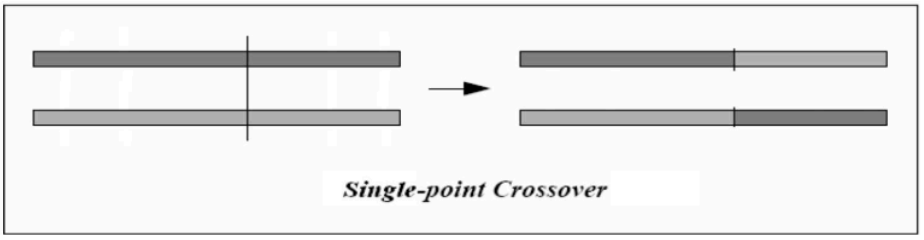
$d=0$ 时为中间重组，一般取 $d=0.25$

子代的每个变量均产生一个 α

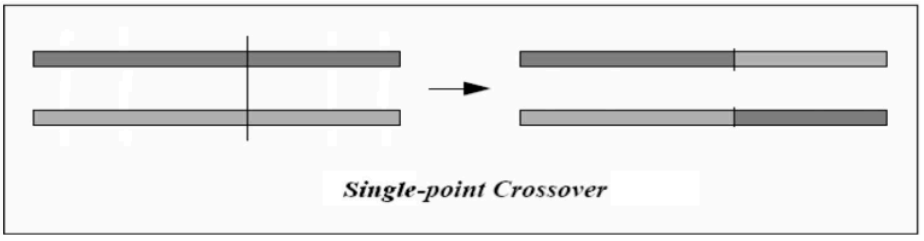
线性重组：每一个子体只有一个 α

二进制交叉

单点交叉



多点交叉



均匀 (一致)交叉：

随机产生 {0,1} 位串, 长度与个体的相等, 称**屏蔽模板**

“1” 的基因座上, 将父代①、②的基因分别传到子个体①、②
“0” 的基因座上, 将父代①、②的基因分别交叉传到子代②, ①

5. 对子代染色体进行**变异**
   实值变异：$X' = X \pm 0.5L\Delta, \Delta = \sum_{i=0}^{m} \frac{a(i)}{2^i}$
   $a(i)$ 以概率 $\frac{1}{m}$ 取值 1 , 以概率 $1 - \frac{1}{m}$ 取值 0 , 通常 m = 20
   L 为变量的取值范围
6. 由交叉和变异产生新一代种群，返回步骤2，直到最优解产生

粒子群优化算法 particle swarm optimization, PSO

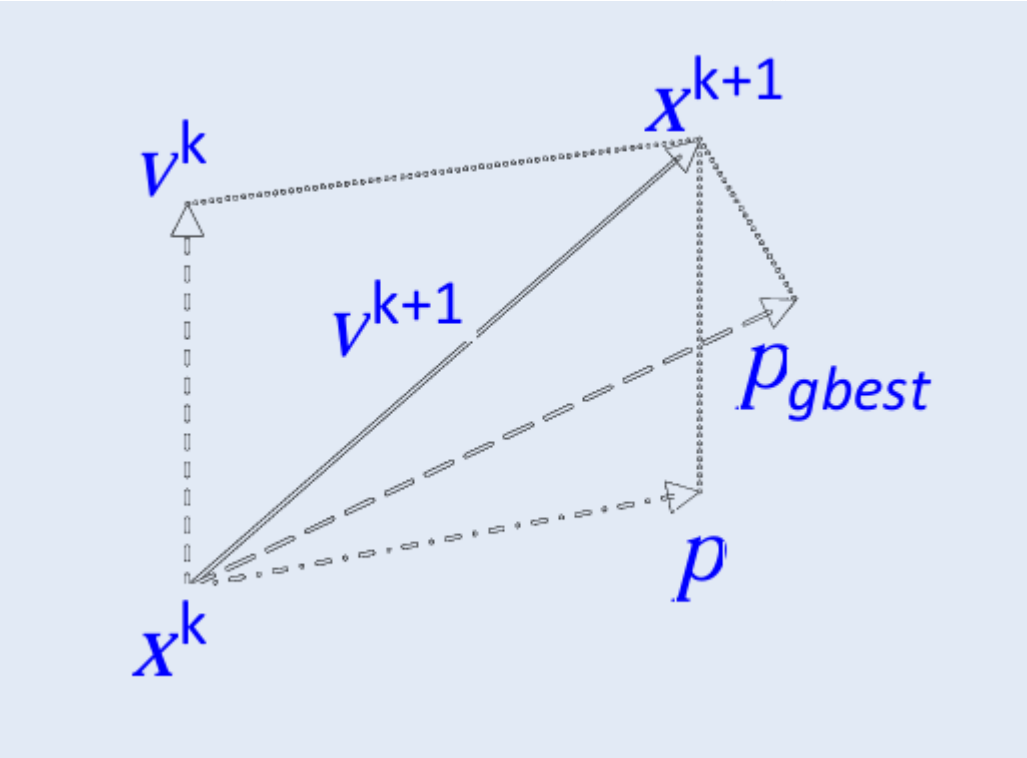
粒子速度和位置的更新

假设在 D 维搜索空间中, 有 m 个粒子;
其中第 i 个粒子的位置为矢量 $\vec{x}_i = (x_{i1}, x_{i2}, \ldots, x_{iD})$
其飞翔速度也是一个矢量，记为 $\vec{v}_i = (v_{i1}, v_{i2}, \ldots, v_{iD})$
第i个粒子搜索到的最优位置为 $\vec{p}_i = (p_{i1}, p_{i2}, \ldots, p_{iD})$
整个粒子群搜索到的最优位置为 $\vec{p}_{gbest} = (p_{gbest1}, p_{gbest2}, \ldots, p_{gbestD})$
第 i 个粒子的位置和速度更新为：

$$v_{id}^{k+1} = wv_{id}^{k} + c_1 rand()(p_{id} - x_{id}^{k}) + c_2 rand()(p_{gbest} - x_{id}^{k})$$

$$x_{id}^{k+1} = x_{id}^{k} + v_{id}^{k+1} \quad , i = 1, 2, \ldots, m; d = 1, 2, \ldots, D$$

其中, W 称为**惯性权重**，c_1 和 c_2 为两个正常数, 称为**加速因子**,将 v_{id}^k 限制在一个最大速度 v_{max} 内



$$v_{id}^{k+1} = wv_{id}^k + c_1 rand()(p_{id} - x_{id}^k) + c_2 rand()(p_{gbest} - x_{id}^k)$$

$$x_{id}^{k+1} = x_{id}^k + v_{id}^{k+1} \qquad i = 1,2,\cdots,m;\ d = 1,2,\cdots,D$$

“惯性部分”，对自身运动状态的信任

“认知部分”，对微粒本身的思考，即来源于自己经验的部分

“社会部分”，微粒间的信息共享，来源于群体中的其它优秀微粒的经验

交换子和交换序

设 n 个节点的TSP问题的解序列为 $s = (a_i), i = 1 \ldots n$.

交换子：$\mathrm{SO}(i_1, i_2)$ 为交换解S中的点 a_{i1} 和 a_{i2} ，则 $\mathrm{S}' = \mathrm{S} + \mathrm{SO}(i_1, i_2)$ 为解S经算子 $\mathrm{SO}(i_1, i_2)$ 操作后的新解。

交换序：一个或多个交换子的有序队列，记作$SS, SS = (SO1, SO2, \ldots SON)$。其中，$SO1, \ldots, SON$ 等是交换子，之间的顺序是有意义的，意味着所有的交换子依次作用于某解上。

交换序的合并算子：若干个交换序可以合并成一个新的交换序，定义 $\oplus$ 为两个交换序的合并算子。

设两个交换序 SS_1 和 SS_2 按先后顺序作用于解 S 上，得到新解 S' 。假设另外有一个交换序 SS' 作用于同一解 S 上，能够得到相同的解 S'

$$SS' = SS_1 \oplus SS_2$$

SS' 和 $SS_1 \oplus SS_2$ 属于同一**等价集**，在交换序等价集中，拥有最少交换子的交换序称为该等价集的**基本交换序**

解决TSP问题的速度算式

$$\begin{cases} \boldsymbol{v}'_{id} = \boldsymbol{v}_{id} \oplus \alpha(\boldsymbol{p}_{id} - \boldsymbol{x}_{id}) \oplus \beta(\boldsymbol{p}_{gd} - \boldsymbol{x}_{id}) \\ \boldsymbol{x}'_{id} = \boldsymbol{x}_{id} + \boldsymbol{v}'_{id} \end{cases}$$

α 、β 为 [0,1] 上的随机数。 V_{id} 表示交换序， x_{id} 表示路径序列（解）

算法流程

1. 初始化粒子群, 给每个粒子一个初始解 (x_{id}) 和随机的交换序 (v_{id})
2. 如果满足结束条件，转步骤5
3. 根据粒子当前位置 x_{id} 计算下一新解 x'_{id}
    1. 计算 $A = p_{id} - x_{id}$, A 是一个基本交换序, 表示 A 作用于 x_{id} 得到 p_{id}
    2. 计算 $\mathrm{B} = p_{gd} - x_{id}$, B 也是一个基本交换序
    3. 更新速度 $v'_{id} = v_{id} \oplus \alpha(p_{id} - x_{id}) \oplus \beta(p_{gd} - x_{id})$,并将其转换为一个基本交换序
    4. 更新解 $x'_{id} = x_{id} + \nu_{id}$
    5. 如果找到一个更好得解, 更新 p_{id}
4. 如果整个群体找到一个更好的解, 更新 P_{gd} , 转步骤2
5. 显示结果

与遗传算法比较

共性

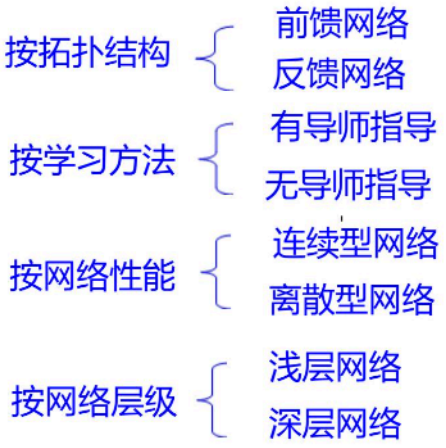
1. 都属于**仿生**算法
2. 都属于**全局优化**方法
3. 都属于**随机搜索**算法
4. 都隐含**并行性**
5. 根据个体的适配信息进行搜索，因此**不受函数约束条件**的限制，如连续性、可导性等
6. 对高维复杂问题，往往会遇到**早熟收敛**和收敛性能差的缺点，都无法保证收敛到最优点

差异

1. PSO有**记忆**，所有粒子都保存较优解的知识，而GA，以前的知识随着种群的改变被改变。
2. PSO中的粒子是一种**单向共享信息**机制。而GA中的染色体之间**相互共享信息**，使得整个种群都向最优区域移动
3. GA需要**编码和遗传**操作，而PSO没有交叉和变异操作，粒子只是通过内部速度进行更新，因此原理更简单、参数更少、实现更容易

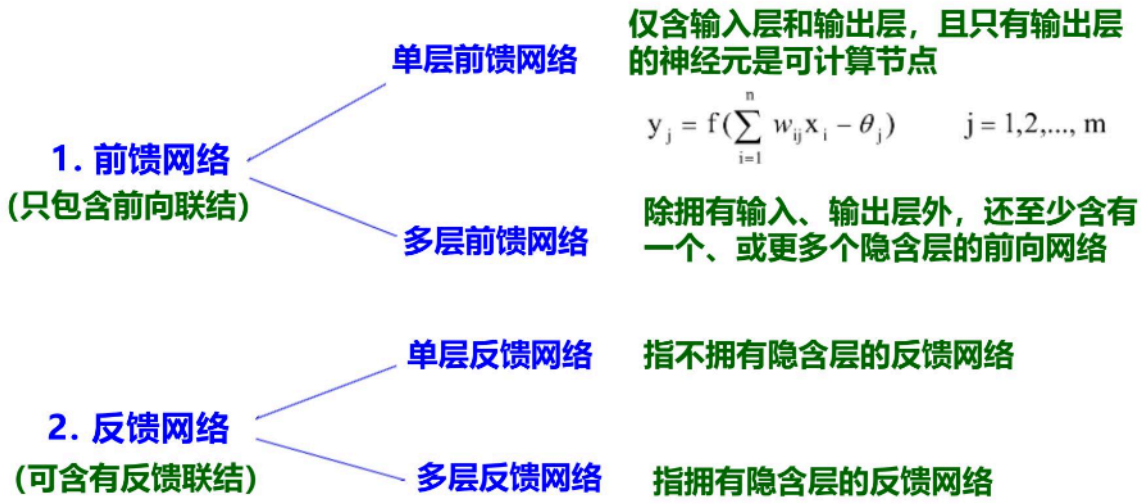
连接学习

人工神经网络的分类



人工神经网络的互连结构（或称拓扑结构）是指单个神经元之间的连接模式，是构造神经网络的基础，也是神经网络诱发偏差的主要来源

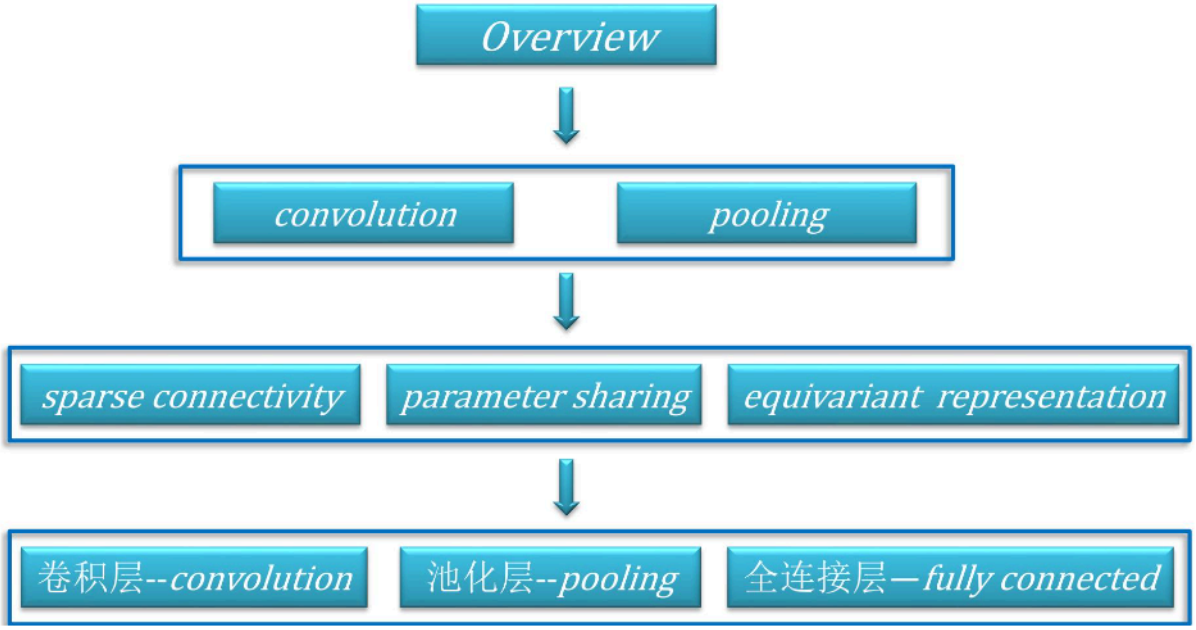
从互联结构的角度，人工神经网络可分为前馈网络和反馈网络两种类型



前向网络: **非循环连接**模式，它的每个神经元的输入都没有包含该神经元先前的输出，因此不具有“短期记忆”的性质

反馈网络: 每个神经元的输入都有可能包含有该神经元先前输出的反馈信息，即一个神经元的输出是由该神经元**当前的输入**和**先前的输出**这两者来决定的，类似于人类的**短期记忆**的性质

深度卷积神经网络



全连接、连接权、稀疏连接、权值共享

卷积操作 (Convolution)

核心思想： CNN的核心操作，用于有效地提取图像特征。
本质： 可以理解为输入矩阵（图像）与卷积核（算子）的乘积。
卷积核： 类似于图像处理中的滤波器（如边缘检测、高斯模糊），对图像进行特征处理。
操作过程： 卷积核在输入图像上滑动（向右或向下），每次滑动都进行一次乘加操作，生成卷积层的一个输出。

稀疏连接 (Sparse Connectivity)

概念： 由于卷积核的大小远小于整幅图像，所以输出层中的每个神经元只与输入层的一个局部区域（由卷积核决定）相连接，而不是与所有输入神经元连接。
对比

全连接： 输出神经元 s_3 受到输入层所有神经元的影响，连接权重覆盖整个输入。

稀疏连接: 以一维视角为例，3x3的卷积核使得输出神经元 s_3 只受到其局部邻域内（例如三个）输入神经元的影响

生物学启发： 模拟生物视觉系统中的局部感知细胞
优点： 减少模型参数，降低计算复杂度，有助于提取局部特征

权值共享 (Weight Sharing)

概念： 同一个卷积核（即同一组连接权重）在输入图像的所有局部区域上重复使用。
意义： 对于图像的不同位置，它们连接到下一层的权重是相同的。
优点： 极大减少了模型的参数数量，使得网络能够学习到与位置无关的特征（即无论特征出现在图像的哪个位置，都能被同一个卷积核检测到）。

池化操作 (Pooling Operation)

目的： 在卷积层之后，通过对图像某块区域进行统计量计算来替代该区域的值。
常见类型：
  Max Pooling (最大池化)： 选取矩形区域内的最大值作为输出。
  Average Pooling (平均池化)： 计算矩形区域内的平均值作为输出。
  Weighted Average Pooling (加权平均池化) 等。
作用： 降低特征图的维度（下采样），减少计算量，同时增加模型的感受野，提高模型的鲁棒性和对平移的少量不变性。

连接学习

Hebb学习规则

如果神经网络中某一神经元同另一直接与它连接的神经元同时处于兴奋状态，那么这两个神经元之间的连接强度将得到加强，反之应该减弱。其对连接权值的调整可表示为:

$$w_{\mathrm{ij}}(t+1)=w_{\mathrm{ij}}(t)+\eta[x_i(t)x_j(t)]$$

- $w_{\mathrm{ij}}(t+1)$ 表示时刻 t 的权值修正一次后所得到的新的权值
- η 是一正常量, 也称为学习因子, 它取决于每次权值的修正量
- $x_i(t)$ 、 $x_j(t)$ 分别表示t时刻第i个和第j个神经元的状态

Hebb学习是**联结学习**中影响较大的一种学习方法，认为对神经元重复同以刺激就可以产生性质相同、程度增强的反应

纠错学习规则

纠错学习也叫误差修正学习，是一种有导师的学习过程
基本思想：利用神经网络的期望输出与实际输出之间的偏差作为连接权值调整的参考，并最终减少这种偏差。
误差修正规则：连接权值的变化与神经元希望输出和实际输出之差成正比。其联结权值的计算公式为:$y_j(t)$ 为神经元 j 的实际输出； $\mathrm{d}_j(t)$ 为神经元 j 的希望输出

$$w_{\mathrm{ij}}(t+1)=w_{\mathrm{ij}}(t)+\eta[\mathrm{d}_j(t)-y_j(t)]x_i(t)$$

学习过程:

1. 选择一组初始 $w_{ij}(0)$
2. 计算某一输入模式对应的实际输出与期望输出的误差
3. 按上述公式更新权值
4. 返回2，直到误差满足要求为止

竞争学习规则

基本思想：竞争中获胜神经元的连接权会向着对其刺激相应模式更为有利的方向发展；而竞争失败神经元的刺激响应模式受到抑制
学习过程：竞争型学习的简单形式是任一时刻只允许一个神经元被激活

1. 将一个输入模式送给输入层LA
2. 将LA层神经元的激活值送到下一层LB
3. LB层神经元对LA层送来的刺激模式进行竞争，每个神经元将一个正信号送给自己（自兴奋反馈），同时将一个负信号送给该层其它神经元（横向邻域抑制）

4. LB层中输出值最大的神经元为竞争获胜神经元，该神经元被激活，连接权值增强；其它神经元为竞争失败神经元，受到抑制，连接权则不变

随机学习规则

基本思想: 结合随机过程、概率和能量（函数）等概念来调整网络的变量，从而使网络的目标函数达到最大（或最小）。
网络变化规则:

1. 如果网络变量的变化能使能量函数有更低的值, 那么就接受这种变化
2. 如果网络变量变化后能量函数没有更低的值, 那么就按某一预先选取的概率分布接受这一变化

随机型学习不仅可以接受能量函数减少（性能改善）的变化，而且还可以以某种概率分布接受使能量函数增大（性能变差）的变化, 模拟退火算法就是一种典型的随机学习算法。

单层感知器学习算法

假设$X(k)$和$W(k)$分别表示学习算法在第k次迭代时输入向量和权值向量，为方便，把阈值θ作为权值向量$W(k)$中的第一个分量，对应地把“-1”固定地作为输入向量$X(k)$中的第一个分量。即$W(k)$和$X(k)$可分别表示如下：

$$X(k)=[-1,x_1(k),x_2(k),\ldots,x_n(k)]$$

$$W(k)=[\theta(k),w_1(k),w_2(k),\ldots,w_n(k)]$$

即$x_0(k)=-1,w_0(k)=\theta(k)$

1. 设 t = 0 , 初始化连接权和阈值。即给 $w_i(0)(i=1,2,\ldots,n)$ 及 $\theta(0)$ 分别赋予一个较小的非零随机数, 作为初值。其中, $w_i(0)$ 是第0次迭代时输入向量中第i个输入的连接权值, $\theta(0)$ 是第0次迭代时输出节点的阈值.
2. 提供新的样本输入$\mathrm{x}_i(t)(\mathrm{i}=1,2,\ldots,\mathrm{n})$ 和期望输出$d(t)$
3. 计算网络的实际输出: $y(t)=f\left(\sum_{i=1}^n w_i(t)x_i(t)-\theta(t)\right) \qquad i=1,2,\ldots,n$
4. 若 $y(t)=d(t)$, 不需要调整连接权值, 转(6)。否则, 需要调整权值
5. 调整连接权值

$$w_i(t+1)=w_i(t)+\eta[d(t)-y(t)]x_i(t) \quad i=1,2,\ldots,n$$

η 是一个增益因子，用于控制修改速度，其值如果太大，会影响 $w_i(t)$ 的收敛性；如果太小，又会使 $w_i(t)$ 的收敛速度太慢；

6. 判断是否满足结束条件，若满足，算法结束；否则，将t值加1，转(2)重新执行。这里的结束条件一般是指 $w_i(t)$ 对一切样本均稳定不变。若输入的两类样本是线性可分的，则该算法就一定会收敛。否则，不收敛。

卷积神经网络 RNN

前馈神经网络，受生物学上感受野（Receptive Field）的机制提出
在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元

结构

例：卷积神经网络的找矿预测模型主要由

- 数据输入层
  网格化的化探和航磁数据作为输入数据写入神经单元

- 卷积层
  选择合适的激活函数完成对数据特征的提取
- 池化层
  选择合适的激活函数完成对数据特征的下采样
- 全连接层
  在网络末端实现特征的映射和分类
- 输出层
  结果输出或特征可视化

结构特性

- 局部连接
- 权重共享
- 空间或时间上的次采样

损失函数

平均绝对误差损失函数（Mean Absolute Error）

也被称为损失函数，用于计算神经网络输出值与真实值之间误差的平均绝对值大小，常用于处理回归问题

$$MAE = \frac{1}{N}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

公式中 y_i 是神经网络预测值， $\hat{y}_i$ 是真实值

均方差损失函数（Mean Squared Error）

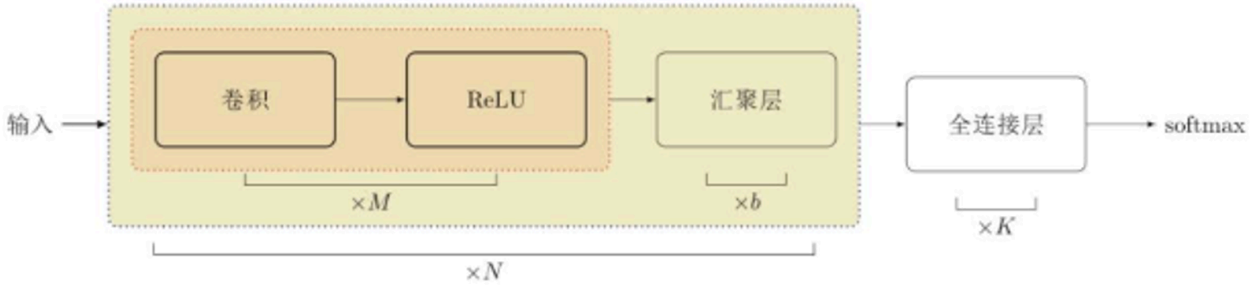
用于度量神经网络输出值与实际值差的平方期望值，均方差损失函数可以很好用于度量数据的变化程度。其公式如下所示：

$$MSE = \frac{1}{N}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

其中， N 表示样本的总量，均方差损失函数是常用于回归问题的损失函数

结构

卷积网络是由卷积层、子采样层、全连接层交叉堆叠而成



一个卷积块为连续M个卷积层和b个汇聚/池化层（M通常设置为2~5，b为0或1）
一个卷积网络中可以堆叠N个连续的卷积块，然后再接着K个全连接层(N的取值区间比较大，比如1~100或者更大；K

一般为0~2)

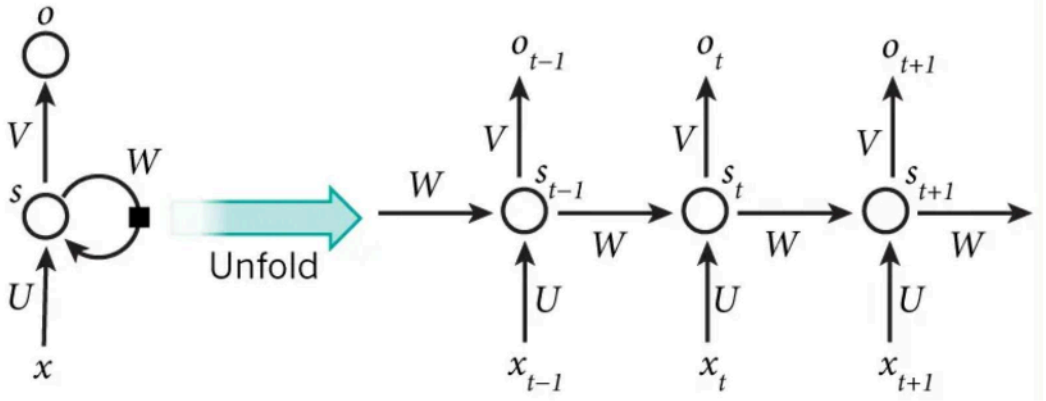
应用

1. 目标检测
   对输入图像进行分类的同时，检测图像中是否包含某些目标，准确定位并标识
2. 光学文字识别OCR
   对文本资料的图像文件进行分析识别处理，获取文字及版面信息
3. 图像生成
   生成准确和流畅的图像描述，而且能够通过可视化展示出它在生成每个词时关注的图片区域
4. 画风迁移
   使用CNN将输入的内容图像与风格图像的目标特征提取，在此基础上输入噪声图像来组合之前提取的目标特征，并结合特征集来生成损失函数，之后在反复的参数迭代中完成对目标迁移图像的调整，最终把内容图像与风格图像二者的特征图结合起来

循环神经网络 RNN

结构图

循环神经网络的当前 t 时间步状态受上一 $t-1$ 时间步状态的影响, 是能处理时序数据的神经网络



结构

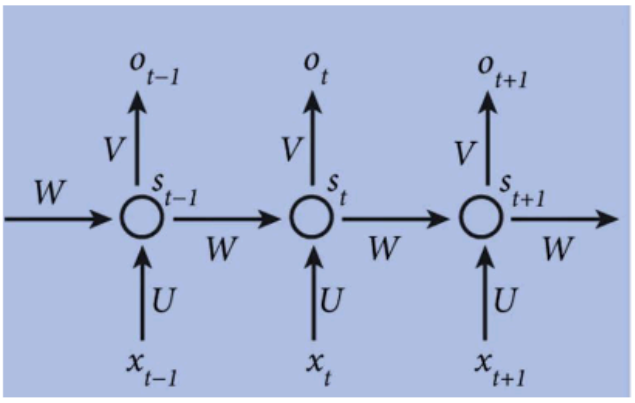
用公式来描述隐状态的计算过程，假设在时刻 t ，网络的输入为 x_t ，隐状态（即隐藏层神经元活性值） h_t 不仅和当前时刻的输入 x_t 相关，也和上一个时刻的隐状态 h_{t-1} 相关，进而与全部过去的输入序列 $(x_1, x_2, \ldots, x_{t-1}, x_t)$ 相关

$$\mathbf{z}_t = U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}, \quad \mathbf{h}_t = f(\mathbf{z}_t)$$

其中 z_t 是隐藏层的净输入; $f(\cdot)$ 是非线性激活函数, 通常为$Sigmoid$函数或$Tanh$函数; U是状态- 状态权重矩阵, W是状态- 输入权重矩阵, b为偏置。
在所有的时刻，使用相同参数和相同的激活函数 $f(\cdot)$

原理

循环神经网络的神经元接**受其他神经元**和**自身**两部分信息，进而形成具有环路的网络结构，循环神经网络的公式如下：



$$O_t = g(V \cdot S_t)$$
$$S_t = f(U \cdot X_t + W \cdot S_{t-1})$$

St的值不仅仅取决于Xt，还取决于St-1

$$\begin{aligned}
&\mathrm{o}_t = g\left(V \mathrm{s}_t\right) \\
&= V f\left(U x_t + W s_{t-1}\right) \\
&= V f\left(U x_t + W f\left(U x_{t-1} + W s_{t-2}\right)\right) \\
&= V f\left(U x_t + W f\left(U x_{t-1} + W f\left(U x_{t-2} + W s_{t-3}\right)\right)\right) \\
&= V f\left(U x_t + W f\left(U x_{t-1} + W f\left(U x_{t-2} + W f\left(U x_{t-3} + \ldots\right)\right)\right)\right)
\end{aligned}$$

计算方法

1. **网络结构：**
   - **输入层维度：2** (即每个时间步输入是一个 `[x1, x2]` 向量)。
   - **隐藏层维度：2** (即隐藏状态是一个 `[s1, s2]` 向量，幻灯片中用 `a1, a2` 表示)。
   - **输出层维度：2** (即每个时间步输出是一个 `[o1, o2]` 向量)。
2. **权重值 (Weights)：**
   - **所有权重值都为 1。** 这意味着图中的 **U、W、V 都是** 2×2 **的全 1 矩阵。**
     - $U = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$
     - $W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$
     - $V = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$
   - **没有偏差 (Bias)。** 实际的神经网络通常有偏置项，这里为简化计算去掉了。
3. **激活函数 (Activation Functions)：**
   - **所有激活函数都是线性函数。** 这意味着如果 $f(x) = x$，$g(x) = x$。所以公式中 $S_t = f(\ldots)$ 和 $O_t = g(\ldots)$ 的 f 和 g 实际上就相当于直接把括号里的内容输出，没有任何非线性变换。
     - $S_t = U \cdot X_t + W \cdot S_{t-1}$
     - $O_t = V \cdot S_t$
4. **初始状态：**
   在第一个输入序列到来前，隐藏层的状态 (S_0) 被初始化为 $[0, 0]$ (幻灯片中说“a1, a2 都没有存值，因此都初始化为0”)。

5. **输入序列：**
    - `[1,1]` (第一个时间步)
    - `[1,1]` (第二个时间步)
    - `[2,2]` (第三个时间步)

时间步 $t = 1$

- **当前输入** $X_1 = [1, 1]^T$ (这里用转置符号 T 表示列向量，便于矩阵乘法)
- **上一时刻隐藏状态** $S_0 = [0, 0]^T$ (初始值)
  根据公式：

1. **计算当前隐藏状态** S_1**：**
   $S_1 = U \cdot X_1 + W \cdot S_0$
    - 计算 $U \cdot X_1$：
      $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} (1 \times 1) + (1 \times 1) \\ (1 \times 1) + (1 \times 1) \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$
    - 计算 $W \cdot S_0$：
      $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} (1 \times 0) + (1 \times 0) \\ (1 \times 0) + (1 \times 0) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$
    - 将两者相加，得到 S_1：
      $S_1 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$
      所以，第一个时刻的隐藏状态是 $[2, 2]$
2. **计算当前输出** O_1**：**
   $O_1 = V \cdot S_1$
    - 计算 $V \cdot S_1$：
      $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} (1 \times 2) + (1 \times 2) \\ (1 \times 2) + (1 \times 2) \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$
      **所以，第一个时刻的输出是 $[4, 4]$。

时间步 $t = 2$

- **当前输入** $X_2 = [1, 1]^T$
- **上一时刻隐藏状态** $S_1 = [2, 2]^T$ (这个 S_1 是从上一个时间步计算得到的)
  根据公式：

1. **计算当前隐藏状态** S_2**：**
   $S_2 = U \cdot X_2 + W \cdot S_1$
    - 计算 $U \cdot X_2$：
      $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} (1 \times 1) + (1 \times 1) \\ (1 \times 1) + (1 \times 1) \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$
    - 计算 $W \cdot S_1$：
      $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} (1 \times 2) + (1 \times 2) \\ (1 \times 2) + (1 \times 2) \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$
    - 将两者相加，得到 S_2：
      $S_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \end{pmatrix}$
      **所以，第二个时刻的隐藏状态是 $[6, 6]$。

2. **计算当前输出 O_2：**
   $O_2 = V \cdot S_2$
   - 计算 $V \cdot S_2$：
     $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 6 \\ 6 \end{pmatrix} = \begin{pmatrix} (1 \times 6) + (1 \times 6) \\ (1 \times 6) + (1 \times 6) \end{pmatrix} = \begin{pmatrix} 12 \\ 12 \end{pmatrix}$
     所以，第二个时刻的输出是 $[12, 12]$**。**

时间步 $t = 3$

- **当前输入** $X_3 = [2, 2]^T$
- **上一时刻隐藏状态** $S_2 = [6, 6]^T$ (这个 S_2 是从上一个时间步计算得到的)
  根据公式：

1. **计算当前隐藏状态 S_3：**
   $S_3 = U \cdot X_3 + W \cdot S_2$
   - 计算 $U \cdot X_3$：
     $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} (1 \times 2) + (1 \times 2) \\ (1 \times 2) + (1 \times 2) \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$
   - 计算 $W \cdot S_2$：
     $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 6 \\ 6 \end{pmatrix} = \begin{pmatrix} (1 \times 6) + (1 \times 6) \\ (1 \times 6) + (1 \times 6) \end{pmatrix} = \begin{pmatrix} 12 \\ 12 \end{pmatrix}$
   - 将两者相加，得到 S_3：
     $S_3 = \begin{pmatrix} 4 \\ 4 \end{pmatrix} + \begin{pmatrix} 12 \\ 12 \end{pmatrix} = \begin{pmatrix} 16 \\ 16 \end{pmatrix}$
     所以，第三个时刻的隐藏状态是 $[16, 16]$**。**
2. **计算当前输出 O_3：**
   $O_3 = V \cdot S_3$
   - 计算 $V \cdot S_3$：
     $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} 16 \\ 16 \end{pmatrix} = \begin{pmatrix} (1 \times 16) + (1 \times 16) \\ (1 \times 16) + (1 \times 16) \end{pmatrix} = \begin{pmatrix} 32 \\ 32 \end{pmatrix}$
     所以，第三个时刻的输出是 $[32, 32]$**。**

最终输出序列
按照时间顺序，这个RNN模型的输出序列是：

- **时间** $t = 1$：$[4, 4]$
- **时间** $t = 2$：$[12, 12]$
- **时间** $t = 3$：$[32, 32]$

模式

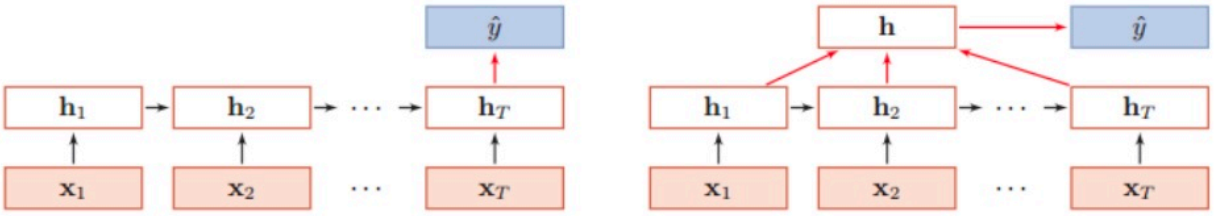
序列到类别的模式

假设有一个样本 $x_{1:T} = (x_1, x_2, \ldots, x_T)$ 为一个长度为 T 的序列，输出为一个类别 $y \in \{1, 2, \ldots, C\}$ 。将样本 $\mathbf{x}$ 按不同的时刻输入到循环神经网络中去，可以得到不同时刻的隐状态 $h_1, h_2, \ldots, h_T$ ，然后将**最后时刻**的隐状态h_T 看做整个序列的最终表示，输入给分类器 $g(\cdot)$ 分类

$$\hat{y} = g(\mathbf{h}_T)$$

还可以对整个序列的所有状态进行平均，用**平均隐状态**来作为整个序列的表示

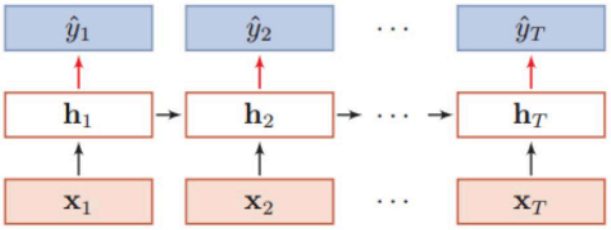
$$\hat{y} = g(\frac{1}{T}\sum_{t=1}^{T}\mathbf{h}_t)$$



(a) 正常模式　　(b) 按时间进行平均采样模式

同步的序列到序列模式

假设有一个样本 $x_{1:T} = (x_1, x_2, \ldots, x_T)$ 为一个长度为 T 的序列，输出序列为 $y_{1:T} = (y_1, y_2, \ldots, y_T)$ 。将样本 $\mathbf{x}$ 按不同的时刻输入到循环神经网络中去，可以得到不同时刻的隐状态 $h_1, h_2, \ldots, h_T$ ，然后把**每个时刻**的隐状态输入给分类器 $g(\cdot)$ ，得到当前时刻的标签$\hat{y}_t = g(\mathbf{h}_t), \quad \forall t \in [1, T]$.



异步的序列到序列模式

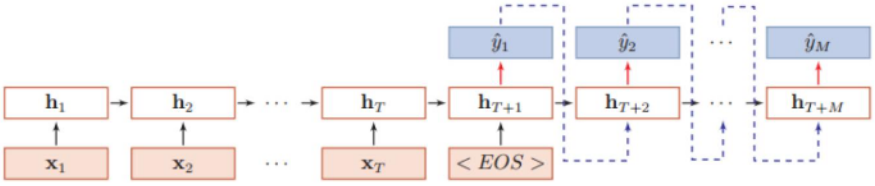
编码器-解码器（Encoder-Decoder）模型，输入序列和输出序列不需要有严格的对应关系，也不用保持相同的长度。比如机器翻译中，输入为源语言的单词序列，输出为目标语言的单词序列。
在异步的序列到序列模式中，输入为一个长度为 T 的序列： $x_{1:T} = (x_1, x_2, \ldots, x_T)$ ，输出一个长度为 M 的序列： $y_{1:M} = (y_1, y_2, \ldots, y_M)$ ，通过先编码后解码的方式实现
先将样本 x 按不同时刻输入到一个循环神经网络（编码器）中，得到其编码 h_t ，然后在另一个循环神经网络（解码器）中得到输出序列 $\hat{y}_{1:M}$ 。为了建立输出序列之间的依赖关系，在解码器中通常使用非线性的自回归模型

$$h_t = f_1\left(h_{t-1}, x_t\right) \quad \forall t \in [1, T]$$

$$h_{T+t} = f_2\left(h_{T+t-1}, \hat{y}_{t-1}\right) \quad \forall t \in [1, M]$$

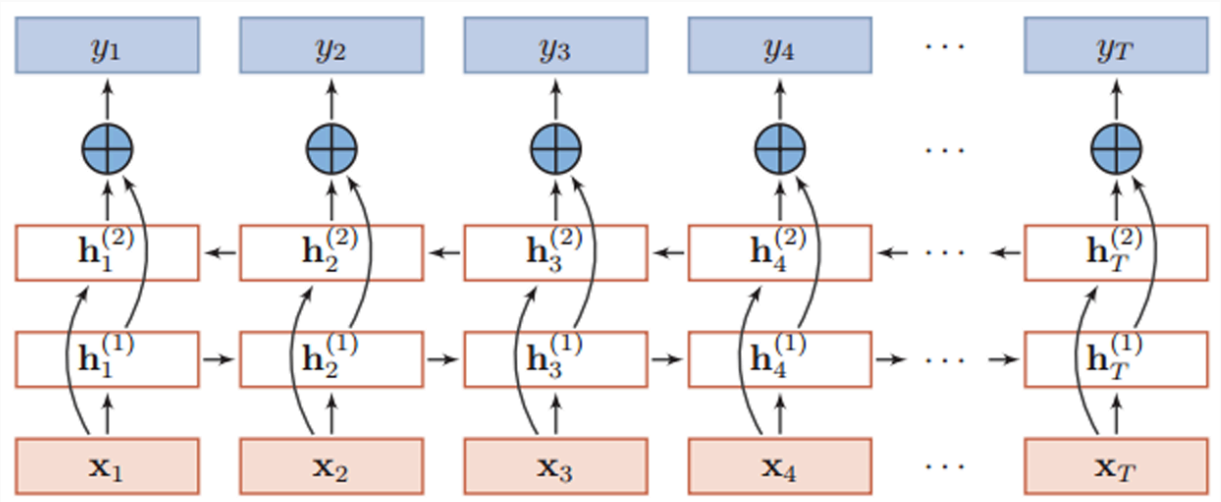
$$\hat{y}_t = g\left(h_{T+t}\right) \quad \forall t \in [1, M]$$

$\mathrm{f}_1(\cdot)$ 和 $\mathrm{f}_2(\cdot)$ 分别表示用作编码器和解码器的循环神经网络， $\mathrm{g}(\cdot)$ 为分类器



双向循环神经网络

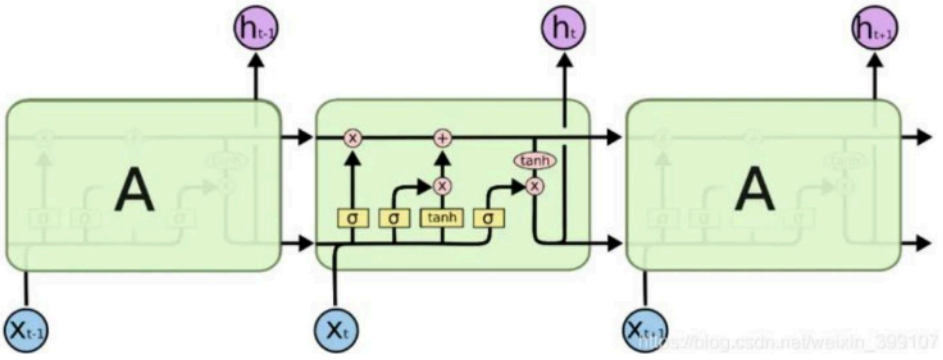
双向循环神经网络按时间展开的图示如下:



可以看到，连接不仅存在于相邻的层与层之间（比如输入层-隐藏层），还存在于时间维度上的隐藏层与隐藏层之间（反馈连接，h_1 到 h_T ）
基本思想是提出每一个训练序列向前和向后分别是两个循环神经网络 （RNN），而且这两个都连接着一个输出层。这个结构提供给输出层输入序列中每一个点的完整的过去和未来的上下文信息。

长短时记忆网络模型 Long Short-term Memory Network

标准 RNN 只有两个主要操作：将先前的**隐藏状态**与**新输入**组合并将其传递给激活函数
LSTM 的控制流程与 RNN 相似，它们都是在前向传播的过程中处理流经的数据，不同之处在于 LSTM 中**细胞的结构和运算**有所变化

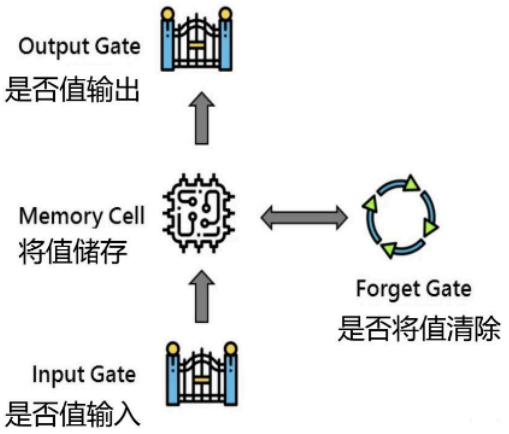


LSTM 门控制机制

LSTM具有**删除或添加信息到细胞状态**的能力，这是门(Gate)结构赋予的。为了控制记忆元，我们需要许多门。
门(Gate): 一种可选地让信息通过的方式, 由**一个Sigmoid神经网络层**和**一个点乘法运算**组成
　　Sigmoid神经网络层输出0和1之间的数字，数字描述每个组件有多少信息可以通过，0表示不通过任何信息，1表示全部通过。
LSTM 具有三(4)个门，以保护和控制单元状态, “是否”通过神经网络进行学习

- Input Gate: 当将feature输入时，input gate会去控制是否将这次的值输入
- Memory Cell: 将计算出的值储存起来，以下个阶段拿出来使用
- Output Gate: 控制是否将这次计算出来的值output

- Forget Gate: 是否将Memory清掉(method), 类似 restart



Forget Gate 遗忘门

LSTM的第一步是决定我们要从细胞状态中丢弃什么信息，由称为“遗忘门”的Sigmoid层实现。它查看前一个输出和当前输入，并为单元格状态(上一个状态)中的每个数字输出0和1之间的数字。1代表完全保留，而0代表彻底删除。
%% 在之前语言模型的例子，细胞状态可能包括当前主语的性别，从而决定使用正确的代词。当我们看到一个新主语时，我们想要忘记旧主语的性别。%%

Input Gate 输入门

决定当前时间步长输入信息的哪些部分需要被更新到当前时间步长的状态信息中。

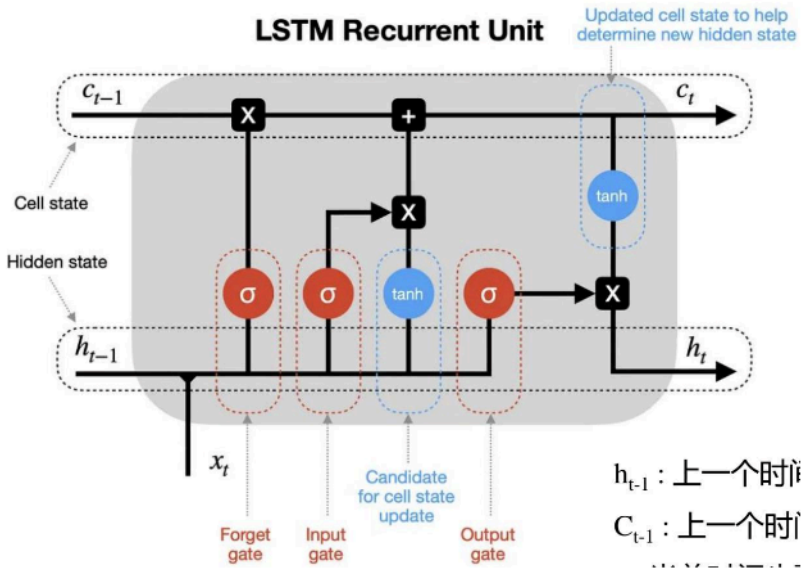
1. “输入门层”的Sigmoid层决定了我们将更新哪些值。
2. 一个tanh层创建候选向量 Ct , 该向量将会被加到细胞的状态中。
   %%在语言模型的例子中，我们希望将新主语添加到单元格状态，替换我们忘记的旧主语%%

Memory Cell 记忆元

长短期记忆网络引入了记忆元（memory cell），或简称为单元（cell）。
长短期记忆网络的隐藏层输出包括“隐状态”和“记忆元”。
只有隐状态会传递到输出层，而记忆元完全属于内部信息。

Output Gate 输出门

首先，运行一个sigmoid层，它决定了我们要输出的细胞状态的哪些部分。再将cell状态通过tanh（将值规范化到-1和1之间），并将其乘以Sigmoid门的输出，至此我们只输出了我们决定的那些部分
%% 在语言模型中，输出的主语是单数还是复数，会影响我们后续输出动词的时态形式%%



h_{t-1} : 上一个时间步骤 t-1 的隐藏状态 (短期记忆)
C_{t-1} : 上一个时间步骤 t-1 的细胞状态 (长期记忆)
$\mathbf{X}_t$ ： 当前时间步骤 t 的输入向量h, h_t 当前时间步骤 t 的隐藏状态
$\mathrm{C_t}$: 当前时间步骤 t 的细胞状态

https://colah.github.io/posts/2015-08-Understanding-LSTMs/